

SUPFile

Documentation Technique

Plateforme de stockage et partage de fichiers dans le cloud

Mai 2026

Sections couvertes

Prérequis & Installation

Guide de déploiement

Justification des choix technologiques

Architecture de l'API

Diagrammes UML

Limites connues & améliorations possibles

Glossaire

Node.js

React / Expo

MongoDB

Docker

Table des matières

1. Prérequis & Installation

- 1.1 Prérequis système
- 1.2 Ports réseau requis
- 1.3 Installation de Docker
- 1.4 Clonage du dépôt
- 1.5 Configuration des variables d'environnement

2. Guide de déploiement

- 2.1 Déploiement Docker Compose
- 2.2 Vérification du déploiement
- 2.3 Déploiement application mobile
- 2.4 Volumes Docker et persistance
- 2.5 Résolution des problèmes courants

3. Justification des choix technologiques

4. Architecture de l'API

- 4.1 Structure du projet backend
- 4.2 Flux d'une requête authentifiée
- 4.3 Principaux endpoints REST
- 4.4 Refresh token — mobile
- 4.5 Sécurité de l'API

5. Diagrammes UML

6. Limites connues & améliorations possibles

7. Glossaire

1. Prérequis & Installation

1.1 Prérequis système

Avant toute installation, assurez-vous que votre machine dispose des éléments suivants :

Outil	Version minimale	Rôle
Docker	≥ 24.0	Conteneurisation des services
Docker Compose	≥ 2.20	Orchestration multiconteneurs
Git	≥ 2.40	Clonage du dépôt source
Navigateur web	Chrome / Firefox / Safari récent	Accès à l'interface web
Node.js	≥ 18 LTS	Lancement Expo (mobile uniquement)
Expo Go	Dernière version	Test de l'app mobile sur téléphone

i Docker Desktop inclut Docker Compose. Une seule installation suffit sur macOS et Windows. Sur Linux, Docker Compose doit être installé séparément : `apt install docker-compose-plugin`.

1.2 Ports réseau requis

Port	Service	Description
4000	Backend API	API REST Node.js / Express
5173	Frontend Web	Interface React (Vite dev server)
27017	MongoDB	Base de données — accès interne Docker (développement : exposé ; production : interne uniquement)
8081	Expo Metro	Bundler React Native + LiveReload

⚠ En développement local, le port 27017 (MongoDB) est exposé par le docker-compose fourni. En production, ne jamais l'exposer en dehors du réseau Docker interne.

1.3 Installation de Docker

macOS

- Télécharger Docker Desktop : <https://www.docker.com/products/docker-desktop/>
- Ouvrir le .dmg et glisser Docker dans /Applications
- Lancer Docker Desktop — attendre l'icône baleine verte dans la barre de menus
- Vérifier l'installation : `docker --version` `docker compose version` **Windows 10 / 11**
- Télécharger et lancer Docker Desktop Installer.exe
- Cocher « Use WSL 2 instead of Hyper-V » durant l'installation
- Redémarrer la machine si demandé
- Vérifier dans PowerShell : `docker --version` `docker compose version`

⚠ WSL 2 doit être activé avec une distribution Linux installée (Ubuntu recommandée via le Microsoft Store).

Sans WSL 2, les performances I/O des volumes sont dégradées.

Linux (Ubuntu / Debian)

```
sudo apt remove docker docker-engine docker.io containerd runc sudo
apt update && sudo apt install -y docker.io docker-compose-plugin sudo
```

```
usermod -aG docker $USER && newgrp docker docker --version && docker
compose version
```

1.4 Structure réelle du projet

Le dépôt est organisé comme suit après clonage :

```
supfile-4proj/
├── supfile-web/
│   ├── backend/      ← API Node.js
│   ├── frontend/     ← App React + Vite
│   └── docker-compose.yml
└── supfile-mobile/   ← App React Native + Expo
```

i docker-compose.yml se trouve dans supfile-web/, pas à la racine du dépôt.
Le mobile (supfile-mobile/) est lancé séparément, en dehors de Docker.

1.5 Configuration des variables d'environnement

Les fichiers .env ne sont jamais versionnés. Des fichiers .env.example sont fournis pour chaque service. Ils contiennent uniquement des valeurs fictives — ne jamais y placer de vrais secrets.

Notion d'adresse IP locale

i localhost fonctionne uniquement depuis la machine qui exécute le service.
Pour tester depuis un téléphone ou un autre appareil du réseau local, remplacer localhost par l'adresse IP locale du PC concerné.

```
IP_PC_BACK = IP du PC qui lance le backend (port 4000)
IP_PC_FRONT = IP du PC qui lance le frontend (port 5173)
```

```
macOS/Linux : ipconfig getifaddr en0
```

```
Windows : ipconfig
```

Backend — supfile-web/backend/.env

```
cp supfile-web/backend/.env.example supfile-web/backend/.env
```

Variable	Valeur / Description
PORT	4000
MONGODB_URI	mongodb://mongodb:27017/supfile (service Docker nommé mongodb)
BASE_API_URL	http://IP_PC_BACK:4000 (URL publique du backend)
FRONTEND_URL	http://localhost:5173 ou http://IP_PC_FRONT:5173

CORS_ORIGINS	http://localhost:5173,http://IP_PC_FRONT:5173
JWT_ACCESS_SECRET	— à définir — (min. 32 caractères)
JWT_REFRESH_SECRET	— à définir — (min. 32 caractères)
ACCESS_TOKEN_TTL	15m
REFRESH_TOKEN_TTL	30d
NODE_ENV	development (local) production
USER_QUOTA_GB	30 (quota de stockage par utilisateur)
MAX_UPLOAD_MB	50 (taille maximale par fichier uploadé)
STORAGE_DIR	/app/storage (dossier de stockage des fichiers)
GOOGLE_CLIENT_ID	OAuth Client ID Google (console.cloud.google.com)
GOOGLE_CLIENT_SECRET	OAuth Client Secret Google — côté serveur uniquement
GOOGLE_CALLBACK_URL	http://IP_PC_BACK:4000/auth/oauth/google/callback
GITHUB_WEB_CLIENT_ID	OAuth Client ID GitHub Web
GITHUB_WEB_CLIENT_SECRET	OAuth Client Secret GitHub Web
GITHUB_WEB_CALLBACK_URL	http://IP_PC_BACK:4000/auth/oauth/github/callback
GITHUB_MOBILE_CLIENT_ID	OAuth Client ID GitHub Mobile (si flux mobile activé)

⚠ Les URLs de callback OAuth (GOOGLE_CALLBACK_URL, GITHUB_WEB_CALLBACK_URL) doivent être enregistrées telles quelles dans les consoles Google et GitHub.

En cas de changement d'IP, mettre à jour ces URLs dans les deux consoles.

⚠ Les fichiers .env ne doivent jamais être versionnés. Vérifier que .gitignore contient bien *.env (et non seulement .env).

Frontend Web — supfile-web/frontend/.env

```
cp supfile-web/frontend/.env.example supfile-web/frontend/.env
VITE_API_URL=http://localhost:4000 # ou http://IP_PC_BACK:4000
```

Application mobile — supfile-mobile/.env

```
cp supfile-mobile/.env.example supfile-mobile/.env
EXPO_PUBLIC_API_URL=http://IP_PC_BACK:4000
EXPO_PUBLIC_GOOGLE_CLIENT_ID=<votre_client_id_google>
```

❗ Les variables EXPO_PUBLIC_* sont intégrées dans le bundle React Native et visibles côté client. Ce sont des Client IDs publics, conformes aux specs OAuth 2.0 pour apps mobiles (flux PKCE). Les Client Secrets restent exclusivement dans backend/.env.

2. Guide de déploiement

2.1 Déploiement complet — Docker Compose

L'ensemble de l'application web (frontend + backend + MongoDB) se déploie depuis le dossier `supfile-web/` :

```
cd C:\supfile-4proj\supfile-web    # Windows cd
~/supfile-4proj/supfile-web        # macOS / Linux
docker compose up --build
```

Cette commande :

- construit les images Docker frontend et backend
- démarre MongoDB avec persistance du volume `mongo_data`
- monte le dossier local `./storage` dans `/app/storage` du conteneur backend • lance l'API sur `:4000` et l'interface web sur `:5173`

Commande	Effet
<code>docker compose up --build -d</code>	Lancement en arrière-plan (mode détaché)
<code>docker compose down</code>	Arrêt propre — données conservées
<code>docker compose down -v</code>	Arrêt + suppression du volume <code>mongo_data</code>
<code>docker compose logs -f backend</code>	Suivi en temps réel des logs backend
<code>docker compose ps</code>	État de tous les conteneurs
<code>docker compose restart backend</code>	Redémarrage isolé du service backend

⚠ `docker compose down -v` supprime le volume `mongo_data` (base de données). Les fichiers uploadés dans `./storage` ne sont PAS supprimés (dossier local). Réserver aux réinitialisations complètes d'environnement.

2.2 Vérification du déploiement

Service	URL de vérification
Frontend Web	<code>http://localhost:5173</code> → page de connexion SUPFile
API Backend	<code>http://localhost:4000</code> → JSON de bienvenue
MongoDB	Port 27017 — accessible uniquement depuis le conteneur backend en production

i La route `/health` n'est pas intégrée par défaut. Pour vérifier rapidement le backend, ouvrir `http://localhost:4000` dans le navigateur : le JSON de bienvenue confirme le démarrage.

2.3 Déploiement de l'application mobile

L'app mobile tourne en dehors de Docker, directement via Expo sur la machine de développement.

i Le mobile ne passe PAS par le frontend web.

Flux web : Navigateur → Frontend Vite :5173 → Backend :4000

Flux mobile : Application Expo → Backend :4000 (directement)

Prérequis :

- Node.js ≥ 18 LTS installé sur la machine
- Expo Go installé sur le téléphone (App Store / Google Play)
- Téléphone et machine sur le même réseau Wi-Fi
- EXPO_PUBLIC_API_URL configuré avec l'IP locale de la machine (IP_PC_BACK)

Lancement :

```
cd C:\supfile-4proj\supfile-mobile # Windows cd
~/supfile-4proj/supfile-mobile # macOS / Linux
npm install npx expo start -c
```

Scanner le QR code avec :

- iOS : Appareil photo natif
- Android : Application Expo Go

2.4 Volumes Docker et persistance

Volume / Chemin	Monté dans	Contenu
mongo_data (volume nommé)	/data/db (conteneur mongodb)	Données MongoDB — utilisateurs, fichiers, partages, tokens
./storage (dossier local)	/app/storage (conteneur backend)	Fichiers physiques uploadés par les utilisateurs

i Sauvegarde MongoDB :

```
docker exec mongodb mongodump --out /data/backup
docker cp mongodb:/data/backup ./mongo_backup
```

Sauvegarde des fichiers uploadés (dossier local ./storage) :

```
cp -r ./storage ./storage_backup # macOS / Linux xcopy
./storage ./storage_backup /E /I # Windows
```


2.5 Résolution des problèmes courants

Symptôme	Cause probable	Solution
Port 4000 ou 5173 déjà utilisé	Autre processus actif	macOS/Linux : <code>lsof -i :4000</code> puis kill Windows : <code>netstat -</code>
		<code>ano findstr :4000 → taskkill /PID <x> /F</code>
Docker ne démarre pas	Docker Desktop non lancé	Vérifier l'icône baleine verte dans la barre système
Mobile ne joint pas l'API	IP incorrecte ou réseau différent	Corriger <code>EXPO_PUBLIC_API_URL</code> avec <code>IP_PC_BACK</code> + même Wi-Fi que le PC
Conteneur en erreur (Exited)	Erreur de config ou dépendance	<code>docker compose logs backend</code> pour voir l'erreur précise
Permissions Docker (Linux)	Utilisateur hors groupe docker	<code>sudo usermod -aG docker \$USER && newgrp docker</code>
Dossier storage manquant	Chemin <code>./storage</code> absent	<code>mkdir -p supfileweb/backend/storage</code> puis relancer <code>docker compose</code>
docker compose introuvable	Lancement depuis la mauvaise racine	Se placer dans <code>supfile-web/</code> avant de lancer <code>docker compose</code>

3. Justification des choix technologiques

Le choix des technologies repose sur cinq critères : sécurité, performance, maintenabilité, compatibilité multiplateforme et évolutivité.

Technologie	Rôle	Justification
Node.js 18+	Serveur d'application	Event Loop non-bloquant — adapté aux I/O intensifs (uploads, accès fichiers)
Express.js	Framework API REST	Routage modulaire, middlewares composables, large écosystème
MongoDB 7	Base de données documents	Schéma flexible (BSON), sans migration coûteuse, indexation native
Mongoose	ODM MongoDB	Schémas stricts, validations, hooks pre/post-save
JWT (HS256)	Authentification stateless	Tokens autoportants, vérifiables sans accès BDD, compatibles web/mobile
OAuth 2.0	Auth déléguée (Google, GitHub)	Délègue la gestion des mots de passe à des fournisseurs de confiance
bcrypt (cost 12)	Hashage mots de passe	Fonction adaptative (~300 ms/hash), résistante aux attaques GPU
Multer	Gestion des uploads	Traitement multipart/formdata, limite de taille configurable (MAX_UPLOAD_MB)
CORS	Sécurité HTTP origines	Whitelist des origines autorisées via CORS_ORIGINS
React 18 + Vite	Frontend web (SPA)	HMR rapide, tree-shaking, composants réutilisables
React Native + Expo	App mobile iOS/Android	Codebase unifiée, accès natif, consomme la même API REST

3.1 Backend — Node.js & Express

Node.js repose sur un modèle événementiel mono-thread (Event Loop) qui délègue les I/O à libuv, permettant à un seul thread de gérer de nombreuses connexions simultanées sans

blocage. Pour SUPFile, dont le cas d'usage principal est le transfert de fichiers, ce modèle est bien adapté.

Le backend reçoit les fichiers via Multer, puis les écrit dans le dossier de stockage (/app/storage). MongoDB ne stocke que les métadonnées (nom, taille, type MIME, chemin relatif storageRelPath).

3.2 Base de données — MongoDB & Mongoose

MongoDB stocke les données sous forme de documents BSON, ce qui permet d'ajouter ou modifier des champs sans migration de schéma. Mongoose ajoute les schémas stricts, les validateurs intégrés et les hooks de cycle de vie (ex. pre('save') pour le hashage du mot de passe).

3.3 Authentification — JWT & OAuth 2.0

Token	TTL	Stockage client web / mobile
Access token	15 min (ACCESS_TOKEN_TTL)	Mémoire JS / Mémoire React Native
Refresh token	30 jours (REFRESH_TOKEN_TTL)	Cookie HttpOnly / exposesecure-store

OAuth 2.0 supporte deux flux distincts :

- Flux Web : le navigateur est redirigé vers Google/GitHub, qui rappelle le backend. Le backend échange le code contre un token, puis redirige vers le frontend avec le JWT.
- Flux Mobile : Expo Auth Session gère le redirect natif. L'app envoie le token IdP au backend, qui le vérifie et retourne le JWT.

3.4 Contrôle d'accès

Chaque ressource est accessible uniquement à son propriétaire (ownerId) ou au bénéficiaire d'un partage interne (targetUserId dans ShareItem). Ce contrôle est appliqué dans les middlewares d'autorisation avant tout accès aux données.

4. Architecture de l'API

L'API suit le paradigme REST avec une séparation des responsabilités en couches : routes, controllers, models et middlewares.

4.1 Structure du projet backend

```
supfile-web/backend/
├── src/
│   ├── controllers/           # Gestion des requêtes/réponses HTTP
│   │   ├── auth.controller.js
│   │   ├── files.controller.js
│   │   ├── share.controller.js
│   │   ├── dashboard.controller.js
│   │   └── user.controller.js
│   └── routes/                # Définition des endpoints REST
│       ├── auth.routes.js
│       ├── files.routes.js
│       ├── share.routes.js
│       └── dashboard.routes.js
```

```

├── user.routes.js
├── middlewares/           # Middlewares transversaux
│   ├── auth.middleware.js # Vérification JWT
│   └── models/           # Schémas Mongoose
│       ├── User.js
│       ├── FileItem.js
│       ├── ShareItem.js
│       └── RefreshToken.js
├── server.js              # Point d'entrée - config Express + MongoDB
├── storage/               # Fichiers uploadés (./storage → /app/storage) ──
├── Dockerfile
└── package.json

```

4.2 Flux d'une requête authentifiée

Étape	Composant	Rôle / Erreur possible
1	CORS middleware	Whitelist origines → 403 si origine non autorisée
2	auth.middleware	Vérification signature + expiration JWT → 401 si invalide
3	Contrôle d'accès	L'utilisateur est-il propriétaire ou bénéficiaire ? → 403
4	Validation	Vérification du payload → 400 Bad Request
5	Controller	Orchestration — traitement de la requête
6	Mongoose / MongoDB	Lecture/écriture → 500 en cas d'erreur

4.3 Principaux endpoints REST

Authentification — /auth

Méthode + Endpoint	Auth.	Description
POST /auth/register	Non	Inscription — email + mot de passe hashé bcrypt
POST /auth/login	Non	Connexion credentials → access token + cookie refresh
POST /auth/refresh	Refresh token	Renouvelle l'access token via le refresh token
POST /auth/logout	Oui	Révoque le refresh token en base (déconnexion sécurisée)

GET /auth/me	Oui	Profil de l'utilisateur connecté
GET /auth/oauth/google	Non	Redirect Google OAuth2 (flux web)
GET /auth/oauth/google/callback	Non	Callback Google — échange code → JWT
POST /auth/oauth/google/mobile	Non	Auth Google mobile — échange token Expo → JWT
GET /auth/oauth/github	Non	Redirect GitHub OAuth2 (flux web)
GET /auth/oauth/github/callback	Non	Callback GitHub — échange code → JWT

i Le flux OAuth GitHub mobile n'est pas finalisé dans la version actuelle.
Seul le flux Google mobile est pleinement fonctionnel.

Fichiers & Dossiers — /files

Méthode + Endpoint	Auth.	Description
GET /files	Oui	Liste les fichiers/dossiers (parentId optionnel)
POST /files	Oui	Upload — multipart/formdata (file + parentId)
GET /files/search	Oui	Recherche par nom (?q=)
GET /files/trash	Oui	Liste les éléments en corbeille (deletedAt != null)
GET /files/breadcrumbs/:id	Oui	Fil d'Ariane jusqu'à la racine
GET /files/:id/preview	Oui	Prévisualisation inline
GET /files/:id/download	Oui	Téléchargement forcé (Content-Disposition: attachment)
GET /files/:id/downloadfolder	Oui	ZIP à la volée d'un dossier
PATCH /files/:id/rename	Oui	Renomme un fichier ou dossier
PATCH /files/:id/move	Oui	Déplace avec détection de cycle

DELETE /files/:id	Oui	Suppression douce → positionne deletedAt
POST /files/:id/restore	Oui	Restaure depuis la corbeille
DELETE /files/:id/hard	Oui	Suppression définitive (fichier physique + MongoDB)

Partage, Dashboard & Utilisateur

Méthode + Endpoint	Auth.	Description
POST /shares/public	Oui	Crée un lien public (token, mot de passe optionnel, expiration)
POST /shares/internal	Oui	Partage interne avec un utilisateur inscrit (email)
GET /shares/with-me	Oui	Liste les éléments partagés avec l'utilisateur
GET /dashboard/usage	Oui	Quota utilisé/libre, répartition par type MIME
GET /dashboard/recent	Oui	Derniers fichiers modifiés ou uploadés
GET /user/me	Oui	Profil complet de l'utilisateur connecté
PATCH /user/me/password	Oui	Mise à jour du mot de passe
PATCH /user/me/avatar	Oui	Upload et remplacement de l'avatar

i BASE_API_URL est utilisée pour générer les liens publics de partage. Si BASE_API_URL pointe vers localhost, les liens ne seront pas accessibles depuis un autre appareil (téléphone, autre machine). Utiliser `http://IP_PC_BACK:4000`.

4.4 Refresh token — mobile

L'application mobile stocke le refresh token dans expo-secure-store (chiffré par le keychain de l'OS). L'access token est conservé en mémoire React Native.

La déconnexion explicite (POST /auth/logout) révoque le refresh token côté serveur, ce qui invalide la session même si le token est intercepté.

i Le mécanisme de refresh automatique (intercepteur Axios jouant les requêtes après un 401) est prévu dans l'architecture. Vérifier sa présence dans `supfile-mobile/src/services/api.ts` avant de le considérer comme pleinement opérationnel dans la version livrée.

4.5 Sécurité de l'API

Couche	Mécanisme	Protection
Transport (production)	HTTPS recommandé	Chiffrement des données en transit — HTTP en local
Origines HTTP	CORS (CORS_ORIGINS)	Blocage des requêtes crossorigin non autorisées
Authentification	JWT HS256 courte durée	Sessions sans état, expiration automatique (15 min)
Sessions longues	Refresh token révocable (BDD)	Révocation explicite à la déconnexion
Autorisation	Contrôle par propriétaire/bénéficiaire	Accès limité au owner ou targetUserId du partage
Mots de passe	bcrypt coût 12	Résistance brute-force, rainbow tables
Uploads	Multer + vérification MIME	Limite de taille (MAX_UPLOAD_MB), filtrage par type

5. Diagrammes UML

5.1 Architecture globale

L'architecture globale comprend trois services conteneurisés (frontend React+Vite, backend Node.js+Express, MongoDB 7), un dossier de stockage local (./storage), et leurs interactions avec les clients web et mobile.

Flux Web : Navigateur → Frontend :5173 → Backend :4000 → MongoDB :27017
 Flux Mobile : Application Expo → Backend :4000 directement (sans passer par le frontend)
 Stockage : Backend → ./storage (dossier local monté dans /app/storage)

5.2 Processus d'upload d'un fichier

Étape	Acteur	Action
1–3	Client (web ou mobile)	Sélection → préparation multipart/form-data → envoi POST /files avec JWT
4–5	auth.middleware	Vérification JWT → 401 si invalide/expiré
6	Controller / validation	Vérification dossier parent + métadonnées → 400/404 si invalide
7	Backend (Multer)	Réception du fichier via Multer → écriture dans /app/storage
8	MongoDB	Création du document FileItem (métadonnées, storageRelPath, parentId)
9–10	Backend → Client	Réponse JSON avec le FileItem créé → rafraîchissement de la liste

5.3 Schéma logique de la base de données

Collection	Champs clés	Relations
User	_id, email, passwordHash, provider, providerId, avatarUrl, avatarMeta, createdAt, updatedAt	1 User → 0..* FileItem / ShareItem / RefreshToken
FileItem	_id, ownerId, type, originalName, mimeType, size, storageRelPath, parentId (auto-ref), deletedAt	parentId → FileItem (arborescence) ; ownerId → User
ShareItem	_id, ownerId, targetUserId, nodeId, nodeType, mode, token, password, expiresAt, allowDownload	ownerId / targetUserId → User ; nodeId → FileItem

RefreshToken	_id, userId, tokenHash, revokedAt, expiresAt, createdAt, updatedAt	userId → User ; révocable via POST /auth/logout
--------------	--	---

5.4 Diagramme des cas d'utilisation

Catégorie	Acteur	Cas d'utilisation principaux
Profil & Données	Connecté	Gérer son profil · Avatar · Changer mot de passe
Connexion	Les deux	S'inscrire · Se connecter (credentials, GitHub, Google)
Gestion fichiers	Connecté	Créer dossier · Uploader · Rechercher · Renommer · Déplacer · Supprimer
Partage	Connecté	Partage interne (email) · Lien public (mot de passe, expiration, téléchargement)
Corbeille	Connecté	Restaurer · Supprimer définitivement · Vider la corbeille
Accès lien public	Non connecté	Vérifier le lien · Prévisualiser · Saisir mot de passe · Télécharger
Autre	Connecté	Thème clair/sombre · Dashboard

6. Limites connues & améliorations possibles

i Cette section liste honnêtement les points non finalisés ou non implémentés dans la version livrée, afin d'éviter tout malentendu lors de l'évaluation.

Point	Détail
Documentation Swagger/OpenAPI	Non intégrée. Les endpoints sont documentés dans cette documentation technique uniquement.
Déploiement production	Non réalisé. Le projet fonctionne en environnement de développement local (HTTP, NODE_ENV=development).
HTTPS	Non configuré en local. Recommandé en production via un reverse proxy (Nginx, Traefik).
Prévisualisation fichiers	Limitée à certains types MIME. Les fichiers non supportés renvoient uniquement le téléchargement.
Refresh automatique mobile	L'intercepteur Axios est prévu dans l'architecture. À vérifier dans api.ts avant de considérer cette fonctionnalité comme opérationnelle.
OAuth GitHub mobile	Non finalisé. Seul le flux Google mobile est pleinement fonctionnel dans la version livrée.
Validation Zod	Non utilisée dans le code actuel. La validation est assurée par les schémas Mongoose et les vérifications dans les controllers.
Helmet	À vérifier dans server.js. La documentation l'annonce ; confirmer sa présence effective.
Rotation des refresh tokens	Non implémentée. Le refresh token est révoquant à la déconnexion, mais il n'y a pas de rotation automatique à chaque renouvellement.
Magic bytes (vérification MIME réelle)	La vérification porte sur le type MIME déclaré par Multer, pas sur les magic bytes du fichier.

7. Glossaire

Terme	Définition
Access Token	JWT de courte durée (15 min) transmis dans l'en-tête Authorization: Bearer. Contient l'id utilisateur, signé HMACSHA256.
BASE_API_URL	URL publique du backend, utilisée pour générer les liens de partage. Doit pointer vers IP_PC_BACK:4000 pour fonctionner hors de la machine locale.
BSON	Binary JSON — format de sérialisation utilisé par MongoDB, supportant des types supplémentaires (ObjectId, Date, Binary).

CORS	Cross-Origin Resource Sharing — mécanisme HTTP permettant ou restreignant les requêtes depuis d'autres domaines.
CORS_ORIGINS	Variable d'environnement listant les origines autorisées (localhost:5173, IP_PC_FRONT:5173, etc.).
deletedAt	Champ de FileItem représentant la suppression douce (soft delete). null = actif, Date = en corbeille.
Docker Compose	Outil d'orchestration multi-conteneurs décrit dans dockercompose.yml. Gère le réseau, les volumes et les dépendances.
Expo Go	Application mobile permettant de prévisualiser une app React Native en développement sans compilation native.
HMR	Hot Module Replacement — rechargement à chaud des modules modifiés sans rechargement complet de la page (Vite).
IP_PC_BACK	Adresse IP locale du PC exécutant le backend sur le port 4000. À utiliser à la place de localhost pour les accès depuis d'autres appareils.
IP_PC_FRONT	Adresse IP locale du PC exécutant le frontend web sur le port 5173.
MONGODB_URI	Variable d'environnement de connexion MongoDB. En Docker : mongodb://mongodb:27017/supfile (nom du service dockercompose).
ODM	Object Document Mapper — Mongoose est l'ODM de MongoDB, équivalent d'un ORM pour les bases relationnelles.
parentId	Référence auto-réursive dans FileItem permettant de construire l'arborescence fichiers/dossiers.
PKCE	Proof Key for Code Exchange — extension OAuth 2.0 pour les clients publics (apps mobiles) sans secret partageable.
Refresh Token	Token longue durée (30 jours) stocké côté serveur (collection RefreshToken). Permet de renouveler l'access token sans reconnexion.
storageRelPath	Chemin relatif du fichier physique dans /app/storage, stocké dans FileItem pour découpler le stockage de l'URL publique.
Contrôle d'accès	Accès à une ressource réservé à son propriétaire (ownerId) ou au bénéficiaire d'un partage interne (targetUserId).